NAMMACART

Build a complete local data platform

A business project from your first SQL query to advanced PySpark, reliable pipelines and senior engineering decisions.

20 SPRINTS	161 TASKS	107,000 ORDERS
Plain-language manager requests	You implement every solution	100,000 baseline + 7,000 later arrivals

Your role

You are the data engineer for a fictional retailer. Your job is to make sales, payment, delivery, inventory and customer reports trustworthy - and keep them trustworthy when data is late, duplicated, malformed or changed.

Included

A complete problem statement, step-by-step sprint assignments, acceptance criteria, folder responsibilities, data dictionary, scenario packs and 1,179,496 physical data records. No cloud account, solution code, completed queries or starter implementation is supplied.

Start with Sprint 00. Build one small, correct result before scaling to the full dataset.

Prepared for hands-on self-study | 21 October 2026

The complete project question

Understand the business before choosing the technology

Your project

NammaCart is a fictional Indian retailer that sells through a website, an app, and physical stores. Its order, payment, warehouse, inventory, and marketing teams export files independently. You are the data engineer asked to turn those files into trusted, repeatable reports on one laptop.

The complete assignment

Design and build a local data platform from scratch. Receive and preserve source files, validate and clean records, create business facts and dimensions, calculate agreed metrics, process new batches and customer changes, support a small event stream, test correctness, measure performance, and recover safely from failures. You write every implementation, query, test, configuration and scheduler yourself.

The outcome

A colleague should be able to follow your instructions, rebuild the same reports, identify rejected records, rerun a batch without changing the answer, and recover from a failed run. The final deliverable is a working project plus evidence that it is correct and maintainable.

Who you are practicing for

Early sprints teach absolute basics. Later sprints ask for the reasoning, operational ownership and tradeoffs often expected in a role requiring five or more years of experience. Finishing a project builds evidence of skill; it does not substitute for five years of operating real production systems.

What is supplied

A ready-made dataset, data dictionary, business rules, twenty sprint briefs, scenario packs and hand-calculated acceptance checks. There is no solution code, starter pipeline, completed SQL, or delivered generator.

Scope

This is a broad end-to-end curriculum, not a claim to cover every tool or every possible production failure. It focuses on Python, SQL, PySpark, modeling, quality, incremental data, CDC, orchestration, testing, performance, streaming and operational design. Cloud accounts are not needed.

What the business must be able to answer

Your final acceptance questions

01. How much eligible merchandise was booked each day, and how much remains after approved returns?

02. Which stores, channels, categories and products contribute the most net merchandise value?

03. How much cash was successfully charged and refunded on each payment date?

04. Which customers are repeat purchasers, and how do purchasing cohorts behave over time?

05. How many delivered orders arrived on time, and which carriers are frequently late?

06. Which products and stores have high return-quantity rates, and why were items returned?

07. Which tracked product/store combinations are out of stock or below their reorder level?

08. How do visitor sessions progress from product view to cart, checkout and purchase?

09. Which reports changed when late files or historical customer corrections arrived?

10. Can you prove the reports are correct, rerunnable, observable and recoverable?

Expected release

A reproducible local project that processes all baseline and daily batches, publishes documented Gold tables, produces these reports, passes business tests, and survives the agreed failure drills.

Boundaries

The supplied data has one currency, no taxes, no ad-spend data, no inventory movements, at most one shipment per order, and approved returns only. Extensions may introduce missing scenarios as isolated fixtures rather than pretending the data contains them.

A local setup that stays manageable

Install the basics first; add tools only when a sprint needs them

Tool	Purpose and reference choice
Python 3.11	Main language for your implementation and small utilities. Use an isolated environment.
Java 17 + PySpark 3.5.7	Reference learning combination. Spark runs in local mode. This is a deliberately fixed curriculum baseline, not a claim that 3.5.7 is the newest release.
Spark SQL	Use the same execution engine as your DataFrame jobs for equivalent transformations and reconciliation.
DuckDB 1.4.5 LTS	Local SQL practice and a lightweight serving database. Use documentation matching this version where syntax differs from the current release.
CSV, JSON Lines and Parquet	Source formats are provided; you create Parquet outputs. SQL types and file formats are separate concerns.
Git and a code editor	Local version history, reviews and reproducible project organization. A hosted repository is optional.
A local test runner and scheduler	Choose tools you can explain. A simple local workflow is enough before learning a larger orchestrator.
Delta Lake 3.3.x, later	Local ACID-table lab in Sprint 18. Select an exact 3.3.x release compatible with Spark 3.5.x and pin it after checking its requirements.
PostgreSQL, Airflow, Docker, Kafka, dbt	Optional follow-on labs. Do not install everything on day one. They add useful skills but are not prerequisites for the core project.

The reference Spark 3.5.7 documentation supports Python 3.8+ and Java 8/11/17; this curriculum narrows that to Python 3.11 and Java 17. Version compatibility is checked against the official sources at the end. Your actual installation and your own implementation still need local validation.

How to work through the project

Practice the habits as well as the syntax

Suggested pace

Budget roughly 220-320 focused hours. One sprint per week is about twenty weeks; take longer for foundations or difficult debugging. Hours are planning estimates, not deadlines.

Start small

Begin with the hand-check files and one or two source files. Create related subsets by selecting orders and all their children, not unrelated random samples of every table. Move to the full data once your logic is stable.

Laptop planning

An 8 GB laptop can begin with small subsets and conservative Spark settings; 16 GB is more comfortable for the full curriculum. Keep roughly 5-10 GB free for extracted sources, several output layers and temporary files. Measure actual use before increasing parallelism.

Operating system

Linux or macOS works well for this reference setup. On Windows, WSL2 is a practical route to a Linux environment. Tool installation may require internet access once; the dataset and routine processing are local afterward.

No cloud required

The source folder stands in for an external delivery location. Your local directories stand in for raw and curated storage. File replay stands in for an event source. These are learning substitutes, not claims of cloud-scale reliability.

Office-style routine

Plan a sprint, break tasks into tickets, work on a feature branch, review changes, run checks, demonstrate the result, and write a short retrospective. If working alone, review your changes after a break as if a colleague wrote them.

Definition of done

A task is complete when its business question is answered, its input/output meaning is documented, meaningful checks pass, rerun behavior is known, and another person can reproduce it. Screenshots alone are not evidence of correctness.

Asking for help

Share the sprint/task ID, what you expected, what actually happened, a small reproducible sample, and the relevant error or plan. Ask for a hint or review first if you want to preserve the learning challenge.

Your sprint roadmap

Every sprint has a request, tasks, evidence and a review question

Sprint	Business outcome	Level
00	Set up your desk like a data engineer	Foundation
01	Learn Python by inspecting real files	Foundation
02	Ask your first SQL questions	Foundation
03	Join tables without changing the meaning	Foundation
04	Meet PySpark and repeat the basics	Foundation
05	Build the raw intake layer	Core
06	Clean data and explain every rejection	Core
07	Understand JSON and business time	Core
08	Build numbers finance can trust	Core
09	Answer time-based and ranking questions	Intermediate
10	Design a warehouse people can use	Intermediate
11	Process new deliveries safely	Intermediate
12	Keep customer history correctly	Advanced
13	Work through advanced SQL and database behavior	Advanced
14	Build tests that catch business mistakes	Advanced
15	Schedule, monitor, and operate the pipeline	Advanced
16	Make Spark faster with evidence	Advanced
17	Process events as they arrive	Advanced
18	Handle table updates, privacy, and recovery	Advanced
19	Deliver and defend the complete system	Senior capstone

Tasks use IDs such as S08-T03. Create your own tickets using these IDs so you can refer to one precise piece of work during a review. Do not skip acceptance checks to move to the next sprint.

Know the delivery you received

Read these rules before loading the data

One dataset, several related tables

A realistic project needs several tables because orders, items, payments, shipments and customer changes have different row meanings. They are delivered as one download.

Baseline

100,000 logical orders placed from January 1 through June 20, 2025 UTC, with source activity available before July 1, 2025 UTC. Customer signups occur in 2024. Inventory is a June 2025 sample.

Seven later batches

batch_date=2025-07-01 through 2025-07-07 each introduces 1,000 new logical orders and related records. Their availability cutoff is the next UTC midnight. In total, 350 orders have old business dates but arrive in these later batches.

Customer changes

customer_cdc exists only in incremental folders. Changes include new customers, full-row updates, soft deletes, duplicates and retroactive corrections. Do not treat a late-arriving effective timestamp as a newer current version automatically.

Physical versus logical counts

The manifest counts physical records, including deliberate duplicates and malformed JSON lines. There are 107,000 unique order IDs after all deliveries. Counts of accepted business rows depend on the specified cleaning and eligibility rules.

File arrival date

Folder batch_date is an arrival-batch label. It is not the customer signup date, order date, payment date or report date. Late records deliberately break that assumption.

File parts

CSV files have a header in every part and use UTF-8. JSON Lines contains one event per line; 21 source lines are deliberately malformed. File splitting is for transfer and processing, not a business partitioning guarantee.

Zero activity

Some batch/table combinations have no records and therefore no file. Use the manifest to distinguish zero expected activity from an expected file that is missing. Do not assume every source must produce rows every day.

Separate test packs

challenge_packs and hand_check are isolated exercises. They are not part of the normal business load. Several challenge files contain case descriptions or smaller schemas, so read their instructions before writing an adapter.

Synthetic data

All people, suppliers, products and activity are invented. Email addresses use example.invalid. Do not contact them. The data is useful for engineering practice, not evidence about real customers or real market behavior.

Dataset inventory

Physical records include replays; they are not final accepted row counts

Source	Baseline rows	Later rows	Total rows
categories	12	0	12
customer_cdc	0	721	721
customers	20,020	0	20,020
inventory	60,000	0	60,000
order_items	300,307	20,838	321,145
orders	100,100	7,007	107,107
payments	115,000	7,720	122,720
products	2,000	0	2,000
returns	5,000	20	5,020
shipments	85,000	782	85,782
stores	20	0	20
suppliers	40	0	40
web_events	425,118	29,759	454,877

Total

1,179,496 physical records across 120 CSV/JSONL files, including 32 isolated challenge/hand-check records. Data-file bytes before compression: 241,330,107 (241.3 MB, decimal units). Supporting JSON metadata and this book are additional.

Relationships

Customers and stores link to orders. Orders link to items, payments and shipments. Products link to items and inventory. Returns link to items; refund payments link to returns. Product categories link to their parent categories. Customer changes link to customer IDs.

Important count

Baseline orders have 100,100 physical records but 100,000 distinct order IDs. Across all arrivals there are 107,107 physical order records and 107,000 distinct order IDs. The difference is deliberate.

Agree on the numbers first

Metric contract / part 1

Money and time

All money is INR in integer paise: 100 paise equals one rupee. Store integers or exact decimals; do not use floating-point money. Store timestamps in UTC and derive Asia/Kolkata dates for business reports. Taxes, foreign exchange, advertising spend and accounting revenue-recognition rules are outside this dataset.

A valid order

An order has a valid unique key, known customer and store, valid timestamp/status, and nonnegative header amounts. Every source item for the order must be present, deduplicated, valid and linked to a known product. Sum of line net plus shipping must equal the header total exactly. An invalid item makes the order ineligible for financial marts until repaired.

Eligible booked order

A valid order with status PAID, SHIPPED or DELIVERED. CANCELLED and PENDING orders are retained for operational analysis but excluded from booked merchandise and eligible order counts. A product being inactive now does not invalidate historical sales.

Line gross and net

Line gross = quantity multiplied by unit_price_paise. discount_paise is the total discount for that entire line, not a per-unit discount or a percentage. Line net = line gross minus line discount.

Order amounts

Booked merchandise = sum of eligible line net. Order payable = merchandise plus shipping_fee_paise, counted once per order. The source order_total_paise is the payable amount even for a pending or cancelled order; status determines reporting eligibility.

Return rules

A return refers to one order item. Count only approved returns with valid positive quantities and amounts. Cumulative returned quantity must not exceed purchased quantity; cumulative refund must not exceed the original line net. Use the supplied refund amount as authoritative within these limits. Shipping is not refunded in this project.

Net merchandise

For a cohort of eligible orders, net merchandise = booked merchandise minus approved refunds on their lines available at the reporting cutoff. Attribute this metric to the original order business date. New returns can therefore restate an earlier order-date report. Call this a training booked-sales measure, not audited accounting revenue.

Net cash

Successful CHARGE amounts minus successful REFUND amounts. FAILED attempts contribute zero. Group cash by payment business date. Keep cash on financially ineligible orders in an exception report rather than silently counting or discarding it; this synthetic main dataset has no unexplained settlement differences after validation.

Use the right denominator and date

Metric contract / part 2

Average order value

Booked merchandise before returns, excluding shipping, divided by the number of eligible booked orders in the chosen period. An empty denominator produces NULL or a clearly labeled no-data value, not a false zero.

Repeat customer share

Among distinct customers with at least one eligible order in the stated observation window, the share with at least two eligible orders in that same window. State the window and cutoff; this is different from cohort retention.

Return quantity rate

Approved returned units associated with the selected eligible order cohort divided by purchased units in that cohort. Do not divide return records by order records and label it a unit-return rate.

On-time delivery rate

Delivered eligible orders with delivered_ts less than or equal to promised_delivery_ts divided by delivered eligible orders, grouped by delivery business date when making a daily delivery report. Exclude undelivered orders from this denominator and report them separately.

Inventory reporting

Inventory covers 500 products, four stores and 30 June dates: 60,000 unique snapshots. A zero means out of stock; a missing product/store/date is not known stock. Quantities can be added across products at one time, not across dates as if they were movements.

Marketing funnel

One source session contributes at most once per step. Require view, cart, checkout and purchase in timestamp order in the same session; break ties by event ID. Attribute the session to the earliest view source. Anonymous sessions are valid. Session-level conversion does not require joining every event to a known customer.

Historical customer attributes

Use the customer version valid at order_ts for historical customer-city reports. Use current attributes only for explicitly current-state reports. SCD intervals are [valid_from, valid_to); a null end denotes the open interval. Soft deletion marks current state inactive and preserves historical facts.

Rules for trust and publication

A rejected source record and a failed pipeline are different things

Preserve evidence

Never edit the supplied raw files to make tests pass. Preserve the original payload, source file, and ingest identity before casting or cleaning.

Deterministic classification

First identify exact replays by the documented raw identity; then parse and validate; then apply parent-key and business checks. Every physical record gets one accounting bucket, although it may have several failed rules. Record a primary reason with a documented priority.

Optional customer attributes

Blank city becomes UNKNOWN; state remains the supplied state unless it also fails your rule. Trim and standardize city case. An invalid email becomes null plus a quality flag. Customer ID, not name or email, identifies the customer.

Required relationships

Unknown customer/store/order/product/item keys are not silently removed by inner joins. Quarantine or park them with a reason. The explicitly artificial orphan IDs C999999 and P99999 never arrive in normal batches.

Known duplicates

Identical payloads with the same key are replays. Different payloads with the same key are not safe to collapse arbitrarily. In the equal-sequence conflicting CDC fixture, flag the conflict and hold that entity for review.

Publication gates

Block publication for unreadable required files, unknown breaking schemas, failed money reconciliation in eligible facts, conflicting published keys, or overlapping customer history. Known invalid source rows may be quarantined while a complete, reconciled valid subset is published with rejection counts.

Eligibility versus quality

A valid pending order is not a defective order. An anonymous event is not a missing customer error. A late delivery is a business performance issue. A malformed JSON line is a technical quality error.

Reference counts

manifest.json provides file checksums, physical counts and selected injected-condition counts. Those counts are useful checks, not a completed cleaned dataset or final business report. New defects you intentionally inject must be recorded separately.

The outputs you must design

Names are suggested; row meaning and correctness are required

Output	Required row meaning / behavior
dim_customer_current	One row per customer ID, including inactive/deleted state; no ambiguous current records.
dim_customer_history	One row per customer validity interval, with a stable version key and non-overlapping time ranges.
dim_product, dim_store, dim_date	One row per product, store, or calendar date; supplier/category dimensions can remain separate.
fact_order	One row per valid order, with explicit booked eligibility and order-level amounts.
fact_order_line	One row per valid order item; financial reports use eligible parents and line-attributed refunds.
fact_payment	One row per valid payment attempt or refund, with success/failure and type retained.
fact_return, fact_shipment	One row per return and one row per shipment. Do not assume every order has either.
fact_inventory_snapshot	One row per snapshot_date, store_id and product_id.
agg_daily_sales	One row per order business date, store and sales channel, with eligible count and merchandise/refund totals.
agg_customer_value	One row per customer for a declared observation window/cutoff; include order count and value.
agg_daily_delivery	One row per delivery business date and carrier with delivered and on-time counts.
agg_session_funnel	One row per session start business date and marketing source, with distinct sessions at each ordered step.
quality_report and run_ledger	One row per defined run/source/rule or file; declare the exact grain before implementation.

Add a reporting cutoff, metric version and publication/run identity to the appropriate reporting metadata. You decide the exact physical schema and key strategy; document the reasons.

Create your own project structure

Folder responsibilities / part 1

Suggested location	What belongs here
README.md	Business purpose, prerequisites, quick start and navigation.
docs/requirements/	Business requests, source contracts, metric definitions and acceptance criteria.
docs/design/	Architecture, data model, lineage and dated design decisions.
docs/runbooks/	Normal operation, rerun, backfill, recovery and incident instructions.
config/	Development/test settings and dependency/version declarations; no secrets.
src/namma_cart/ingestion/	Your source readers and Bronze jobs.
src/namma_cart/transforms/	Your reusable cleaning, validation, joining and history logic.
src/namma_cart/jobs/	Your runnable jobs that coordinate reusable transformations.
src/namma_cart/quality/	Your validation and reconciliation rules.
src/namma_cart/common/	Shared configuration and logging helpers, only where genuinely reused.
sql/	Your query exercises, warehouse definitions and analyst-facing queries.
tests/unit/, tests/integration/	Small transformation checks and complete-flow checks.

This is a folder plan, not a prebuilt repository. Create it yourself in Sprint 00. Filenames and module boundaries may evolve as long as responsibilities remain clear.

Keep source, state and output separate

Folder responsibilities / part 2

Suggested location	What belongs here
tests/fixtures/	Your tiny deterministic test data and independently calculated expected results.
orchestration/	Your dependency graph, runner or scheduler definitions.
notebooks/	Optional exploration; production jobs must also work without an open notebook.
data/source/	The extracted supplied data, preserved unchanged and excluded from Git.
data/bronze/, data/silver/, data/gold/	Your raw, cleaned, and report-ready outputs; excluded from Git.
data/quarantine/, data/staging/	Rejected records and unpublished candidate output; excluded from Git.
data/checkpoints/, data/stream_inbox/	Streaming state and newly arriving files; excluded from Git.
artifacts/reports/, artifacts/metrics/	Your demo reports, reconciliation evidence and run measurements.
logs/	Local execution logs; excluded from Git and scrubbed of direct identifiers.

What belongs in version control

Your source code, SQL, small test fixtures, dependency/version records, contracts, documentation and safe configuration examples.

What stays local

The supplied bulk dataset, produced data layers, credentials, local environments, logs, checkpoints and large temporary files. A small checked-in fixture is different from committing the whole dataset.

Sprint 00 | Set up your desk like a data engineer

Foundation / Planning estimate: 6-8 focused hours

The request. Your manager says: Before you touch sales data, make sure another teammate could open your project and understand where everything belongs.

Use these inputs. Project brief, folder map, baseline customers and products.

What you learn. Terminal, paths, Python environments, Git, README, configuration and reproducibility.

S00-T01 Create the folder structure shown in this book yourself. Explain the purpose of each folder in one sentence. Keep source files separate from files your pipeline produces.

S00-T02 Install the reference tools and record their exact versions and your laptop specifications. Create one isolated Python environment for this project.

S00-T03 Initialize a local Git repository. Create a feature branch, make a small documentation change, inspect the difference, and merge it. Deliberately create and resolve one harmless text conflict.

S00-T04 Write a README explaining the business problem, required tools, where the data lives, and the order in which a new person should run your future jobs.

S00-T05 Create development and test settings with different input and output locations. Keep passwords, generated data, checkpoints, and local logs out of Git.

S00-T06 Write a one-page requirements note: who needs the reports, which questions they ask, what one row means in each source, and what is still an assumption.

S00-T07 Create a ticket for each task in this sprint. Move tickets through To do, Doing, Review, and Done; write a short review checklist. A local text board is enough.

S00-T08 Demonstrate that you can find a file, inspect its first few rows, check its size, and recognize CSV versus newline-delimited JSON without opening the entire dataset in a spreadsheet.

What to hand over. Your own repository, setup record, README, folder map, first tickets, and a short requirements note.

Done when. A fresh terminal can find the intended environment and files. Generated data is excluded from Git. No business transformation code is required yet.

Be ready to explain. What would fail if another person copied your repository to a different folder?

Sprint 01 | Learn Python by inspecting real files

Foundation / Planning estimate: 8-12 focused hours

The request. Your team receives a folder of files. Tell us what arrived and whether any files look suspicious before we process them.

Use these inputs. baseline/customers, baseline/products, manifest.json; then one orders file.

What you learn. Variables, types, conditions, loops, lists, dictionaries, functions, exceptions, files, modules and command-line arguments.

S01-T01 Inspect ten customers and describe strings, integers, booleans, empty values, and timestamps in ordinary language. Explain why a CSV does not itself enforce these types.

S01-T02 Read a CSV correctly even when a customer name contains a comma or quotation marks. Count records without loading the whole file into a Python list.

S01-T03 Write small reusable functions that count rows, list headers, and summarize missing fields. Give each function one clear job.

S01-T04 Produce a file inventory containing relative path, extension, size, row count, and last observed processing status. Compare counts with manifest.json.

S01-T05 Accept an input directory and report location as arguments. Report an understandable error for a missing directory, unreadable file, or unexpected header.

S01-T06 Read nested JSON from a small valid sample. Explain the difference between a missing key, a JSON null, an empty string, and an empty list.

S01-T07 Use the hand_check files to calculate one order total on paper first, then independently reproduce the calculation with your own Python.

S01-T08 Split reusable functions from the file that starts the program. Add clear logging and a short explanation of how to run your work.

What to hand over. A file-inspection utility, inventory report, and notes explaining the Python concepts you used.

Done when. Counts exclude CSV headers. Quoted customer names remain one field. A bad file produces an error report instead of silently disappearing.

Be ready to explain. Why is line-by-line reading different from reading all rows into memory?

Sprint 02 | Ask your first SQL questions

Foundation / Planning estimate: 8-12 focused hours

The request. A business analyst wants simple customer and product lists without manually filtering files.

Use these inputs. Customers, products, stores, suppliers; use a local DuckDB database.

What you learn. Tables, types, SELECT, aliases, WHERE, AND/OR, IN, BETWEEN, LIKE, DISTINCT, ORDER BY, LIMIT, NULL and CASE.

S02-T01 Create typed raw tables from the supplied column descriptions. Preserve the original values; do not quietly clean or discard records during loading.

S02-T02 List the first 20 products in a stable order and show their prices in rupees. Explain why paise should remain the stored money unit.

S02-T03 Find customers in Hyderabad or Bengaluru, customers with blank cities, and customers whose email is missing or invalid. Define your simple email validation rule.

S02-T04 List products priced between 500 and 1,500 rupees, inactive products, and products belonging to a chosen supplier.

S02-T05 Create price bands using a conditional expression. State which band contains a price exactly on a boundary.

S02-T06 Count physical customer rows and distinct customer IDs. Explain why those numbers differ.

S02-T07 Demonstrate the difference between an empty string and SQL NULL. Explain why equality to NULL does not mean a value is missing.

S02-T08 Save each answer as a separately named query you wrote. Add the business question, input tables, expected columns, and sort order as comments.

What to hand over. At least eight beginner queries and a short data-types and missing-values note.

Done when. No query depends on accidental row ordering. Money conversions do not change source amounts. Duplicate customers are visible in your findings.

Be ready to explain. When do you need DISTINCT, and when might it hide a data problem?

Sprint 03 | Join tables without changing the meaning

Foundation / Planning estimate: 8-12 focused hours

The request. The merchandising team wants product, supplier, and order summaries, but some joins accidentally increase the totals.

Use these inputs. Baseline dimensions, orders, order_items and payments; hand_check and JOIN_MULTIPLICATION fixture.

What you learn. Primary and foreign keys, grain, joins, aggregation, HAVING, subqueries, CTEs, UNION and UNION ALL.

S03-T01 Draw the key relationships. For every source, say what one row represents and whether an ID can appear more than once.

S03-T02 Show each product with its category and supplier. Keep unmatched products visible and count them instead of silently losing them.

S03-T03 Count orders per store and per status. Filter grouped results to show stores with more than a chosen number of orders.

S03-T04 Find customers with no orders using two different approaches. Explain why a nullable exclusion list can produce surprising answers.

S03-T05 Compare inner, left, full, semi, and anti joins in plain language. Implement supported forms in your chosen SQL engine; document an equivalent when syntax differs.

S03-T06 Use the JOIN_MULTIPLICATION fixture to show why joining several item rows to several payment rows can overstate sales. Propose the correct intermediate grain before writing your final query.

S03-T07 Combine two daily order batches and compare keeping all rows with removing duplicate rows. Explain why full-row deduplication is different from business-key deduplication.

S03-T08 Use a CTE to make a multi-step query readable. Identify any provisional totals that still include source defects; do not call them trusted revenue yet.

What to hand over. A relationship diagram, join exercises, aggregation queries, and a written explanation of duplicate multiplication.

Done when. The multiplication fixture has a merchandise total of 300 paise, regardless of the number of payment rows. Unmatched keys are counted explicitly.

Be ready to explain. Why can a query return the right number of orders but the wrong revenue?

Sprint 04 | Meet PySpark and repeat the basics

Foundation / Planning estimate: 8-12 focused hours

The request. The same data work must now run through Spark so the team can later process larger volumes.

Use these inputs. Customers, products and one orders CSV part; then all baseline orders.

What you learn. SparkSession, DataFrames, schemas, transformations, actions, lazy evaluation, partitions, driver and tasks.

S04-T01 Create your first local Spark session and document the master setting, available cores, session timezone, and shuffle settings.

S04-T02 Read a small file first with inference, then with a schema you wrote. Compare the results and explain what happens to a bad timestamp.

S04-T03 Select, rename, filter, sort, add, and remove columns. Reproduce five of your earlier SQL questions using the DataFrame API.

S04-T04 Explain which operations describe work and which operations actually start work. Observe the difference in the Spark user interface.

S04-T05 Register a DataFrame for Spark SQL and answer the same question with SQL. Compare results by keys and values, not row position.

S04-T06 Read every orders part. Show only a small preview; calculate large summaries in Spark. Explain the risk of bringing the entire dataset to the driver.

S04-T07 Save a temporary result as Parquet and read it back. Compare its schema and size with the CSV input.

S04-T08 Write a tiny RDD exercise to understand the lower-level abstraction, then explain why your main project uses DataFrames and built-in expressions.

What to hand over. Your first Spark exploration, five SQL/DataFrame comparisons, Parquet round-trip evidence, and an architecture note.

Done when. All baseline order files are read. You can explain driver versus task work. Your solution does not depend on collecting the full dataset.

Be ready to explain. Why can a script appear to do nothing until an action is requested?

Sprint 05 | Build the raw intake layer

Core / Planning estimate: 10-14 focused hours

The request. Every incoming record must be traceable to the file we received, including records that cannot be parsed.

Use these inputs. All baseline sources; manifest.json; header-only and wrong-header challenge files.

What you learn. Bronze ingestion, source contracts, file manifests, metadata, schema enforcement and auditability.

S05-T01 Design the Bronze layer as an unchanged record of what arrived. Choose how to preserve malformed JSON and its original file and line identity.

S05-T02 Define expected columns, raw and target types, required fields, key columns, and allowed values for each source. Treat the supplied dictionary as the starting contract.

S05-T03 Add source file, ingestion run ID, received batch, load time, and source schema version where applicable. Keep source business timestamps separate.

S05-T04 Write a file-level ingestion ledger with checksum, record count, status, start/end time, and error summary. A failed load must remain visible.

S05-T05 Distinguish a valid empty file, a file with only a header, a missing expected file, and a file whose header changed. Document whether each blocks publication.

S05-T06 Store a Bronze representation in Parquet while retaining raw originals. Explain what metadata you need to reproduce the same parse later.

S05-T07 Verify every supplied data-file hash and record count. Count malformed JSON lines as received records, even though they are not valid events.

S05-T08 Design how the same physical file will be recognized if delivered twice under different names. Explain the limitation of filename-only tracking.

What to hand over. Bronze ingestion jobs you wrote, source contracts, file ledger, and a reconciliation report.

Done when. Every input record is accounted for. Bad JSON remains inspectable. A wrong header is reported, and no downstream job treats an incomplete batch as ready.

Be ready to explain. How would you prove which original file produced an incorrect dashboard value?

Sprint 06 | Clean data and explain every rejection

Core / Planning estimate: 10-16 focused hours

The request. The reporting team needs trustworthy rows and a clear explanation of what was rejected.

Use these inputs. Bronze tables, built-in defects, amount_and_quantity_errors challenge pack.

What you learn. Silver processing, parsing, null handling, normalization, deduplication, validation and quarantine.

S06-T01 Normalize customer city whitespace and case. Keep missing cities as UNKNOWN and report them; invalidate bad email values without throwing away an otherwise usable customer.

S06-T02 Parse timestamps and numeric fields explicitly. Quarantine mandatory-field failures, unknown required parent keys, invalid statuses, and impossible amounts or quantities.

S06-T03 Remove exact duplicates deterministically. Separate duplicate records from conflicting versions of the same business key; a random survivor is not an acceptable policy.

S06-T04 Create a rejection table with source identity, business key where available, original payload, primary rejection reason, all failed rules, and run ID.

S06-T05 Reconcile physical input records into mutually exclusive accepted, duplicate, and rejected buckets. Explain the order of checks so one record is not counted twice.

S06-T06 Build a separate order eligibility flag for financial reporting. A missing or rejected item must not silently reduce an order total and leave the order eligible.

S06-T07 Reject over-refunds, excessive return quantities, negative prices, and discounts above the line gross amount in the isolated fixtures.

S06-T08 Publish a quality report by source and rule. Show accepted and rejected percentages with the denominator clearly stated; do not repair unknown IDs by guessing.

What to hand over. Silver datasets, quarantine outputs, quality rules, order eligibility logic, and a row-accounting report.

Done when. Accepted + duplicate + rejected equals physical input at the classified stage. Every financially eligible order passes the header-to-items check. Optional city/email issues do not erase orders.

Be ready to explain. Which errors are repairable, which require quarantine, and which require a conversation with the source owner?

Sprint 07 | Understand JSON and business time

Core / Planning estimate: 8-12 focused hours

The request. Marketing sends nested website events, while finance wants dates based on Indian business time.

Use these inputs. web_events, schema version 2 in July 4-7 batches, schema_break and DST_FALLBACK fixtures.

What you learn. Structs, arrays, maps, JSON parsing, explode, event time, receipt time, timezones and schema evolution.

S07-T01 Write an explicit event schema and flatten device and marketing fields. Preserve the original event ID and session ID.

S07-T02 Expand the cart array into one row per cart entry. Show how empty arrays and null values affect row counts; preserve the event when that is required by the report.

S07-T03 Count events, distinct events, sessions, and anonymous events separately. Explain why they measure different things.

S07-T04 Keep storage timestamps in UTC and derive an Asia/Kolkata business date for reporting. Find events whose UTC date and business date differ.

S07-T05 Compare event time with received time. Measure source delivery delay and explain why this historical bulk extract is not a realistic live-stream ordering.

S07-T06 Accept the optional app_version field added in schema version 2. Test that older events still work and document what should happen to an unexpected breaking type change.

S07-T07 Handle malformed JSON without silently dropping it. Test the unexpected cart string and a far-future event timestamp from the challenge pack.

S07-T08 Use the two UTC instants in the daylight-saving fixture to explain why a local wall-clock time can be ambiguous. Do not change the main project timezone.

What to hand over. Clean event and cart-entry tables, timezone tests, and a schema-change policy.

Done when. Exploding carts does not inflate the event-count report. Old and new event schemas work together. Corrupt payloads remain traceable.

Be ready to explain. Why are event time, arrival time, and processing time three different clocks?

Sprint 08 | Build numbers finance can trust

Core / Planning estimate: 10-16 focused hours

The request. Finance asks: What did we sell, what was refunded, and how much money did we actually collect?

Use these inputs. Clean orders, items, payments, returns and shipments; the metric contract and hand_check.

What you learn. Safe multi-table joins, decimal arithmetic, conditional aggregation, reconciliation and financial grain.

S08-T01 Implement the eligibility and money rules from the metric contract. Keep amounts in integer paise or exact decimal arithmetic through every calculation.

S08-T02 Calculate line gross, line discount, line net, order merchandise, shipping, and order payable amounts. Reconcile order payable to the order header.

S08-T03 Aggregate charge and refund payments independently at order grain before attaching them to orders. Ignore failed attempts when calculating cash.

S08-T04 Produce daily booked merchandise, approved return deductions, net merchandise, and net cash. Explain why the cash report groups by payment date instead of order date.

S08-T05 Calculate average eligible order value, repeat customer share, return quantity rate, and on-time delivery rate using the exact denominators in this book.

S08-T06 Show products and stores with the highest net merchandise value. Allocate refunds to their returned lines, and do not copy order shipping charges onto every line.

S08-T07 Find orders with a successful payment but no delivery, deliveries after the promised timestamp, and orders excluded from financial reports. Treat business states separately from data defects.

S08-T08 Reproduce the hand_check acceptance values. Repeat the core daily sales result once in DataFrame operations and once in Spark SQL; compare every grouping key and amount.

What to hand over. Reconciled financial datasets, at least six business reports, a metric dictionary, and comparison evidence.

Done when. Hand-check net merchandise is 19,000 paise and net cash is 24,000 paise. Failed attempts do not increase collections. Finance totals survive joining extra descriptive fields.

Be ready to explain. Why can sales, approved refunds, and cash legitimately differ on the same day?

Sprint 09 | Answer time-based and ranking questions

Intermediate / Planning estimate: 10-14 focused hours

The request. Managers want trends and customer behavior, not only totals.

Use these inputs. Eligible order/item facts, customer history candidates, date dimension and event sessions.

What you learn. Window functions, ranking, running totals, LAG/LEAD, frames, rolling periods, retention and session boundaries.

S09-T01 Rank products within each category by net merchandise. Show how ROW_NUMBER, RANK, and DENSE_RANK behave on ties, and apply an explicit product-ID tie breaker when needed.

S09-T02 Find each customer's first and most recent eligible order, previous order date, and days between orders. Break equal timestamps using order ID.

S09-T03 Build a complete daily calendar and calculate running sales and trailing seven-calendar-day sales, including dates with zero sales.

S09-T04 Calculate month-over-month change with a documented result for a missing previous month or a zero denominator.

S09-T05 Create a monthly cohort table based on the customer's first eligible purchase and measure later-month repeat purchases. Distinguish a zero from a month not yet observed.

S09-T06 Find streaks of dates with stockouts for each inventory product/store pair. A missing inventory snapshot is unknown, not automatically zero inventory.

S09-T07 Rebuild sessions from events using a 30-minute inactivity threshold as an experiment. Compare these inferred sessions to source session IDs and document anonymous-user limitations.

S09-T08 Define a complete view-to-cart-to-checkout-to-purchase funnel with ordered steps in the same source session. Attribute each session to its earliest view's source; use unknown when absent.

What to hand over. Ranking, calendar, customer, cohort, stockout, and funnel analyses with boundary-case tests.

Done when. A seven-day window is seven calendar days, not seven arbitrary rows. Cohort rates are bounded. Tied ranks and missing dates match the fixtures.

Be ready to explain. When would a ROWS frame answer a different question from a time-range frame?

Sprint 10 | Design a warehouse people can use

Intermediate / Planning estimate: 10-14 focused hours

The request. Analysts want stable tables with clear meanings, so each new report does not rebuild all joins.

Use these inputs. Silver tables and the required Gold output list.

What you learn. Dimensional modeling, facts, dimensions, surrogate keys, grain, star schemas and semantic consistency.

S10-T01 Design a star schema with order, order-line, payment, return, delivery, and inventory grains. Explain which facts can be added across dates and which cannot.

S10-T02 Create customer, product, store, supplier/category, and date dimensions. Choose stable keys and document how unknown members and inactive products behave.

S10-T03 Build the required Gold tables and record each table's primary key, row grain, refresh rule, money unit, and owner.

S10-T04 Separate an order-level fact from an item-level fact. Explain how an analyst should use shipping fees and order counts without multiplying them.

S10-T05 Serve a small set of approved Gold tables in DuckDB and query them with SQL. Keep the source CSV load distinct from the trusted serving layer.

S10-T06 Create report-facing views or tables that expose consistent business names and omit direct customer email and name fields when they are not needed.

S10-T07 Write a source-to-report map for net sales, on-time delivery, and return rate. Identify which source fields can change each result.

S10-T08 Review two alternative models and write a decision record explaining why you chose one. Include a sample analyst question that would be awkward in the rejected design.

What to hand over. Warehouse diagram, Gold tables, serving layer, field dictionary, and a modeling decision record.

Done when. Each Gold table has one declared grain and unique key. Descriptive joins preserve fact totals. Inventory snapshots are not summed as if they were daily sales.

Be ready to explain. What happens when an analyst joins two fact tables directly on customer ID?

Sprint 11 | Process new deliveries safely

Intermediate / Planning estimate: 10-16 focused hours

The request. New files arrive every day. Process only what is new, and make reruns safe.

Use these inputs. Baseline plus incremental/batch_date=2025-07-01 through 2025-07-07; 350 late orders.

What you learn. Incremental loads, idempotency, checkpoints, watermarks, backfills, partition repair and publication.

S11-T01 Load the baseline once, then process the seven incoming batches in arrival order. Record both arrival date and business date.

S11-T02 Design a successful-run ledger and file checksum checks. A rerun of the same batch must not duplicate Bronze records or business results under your documented ingestion contract.

S11-T03 Demonstrate why filtering only for order timestamps newer than the last run misses late orders. Choose an arrival-based intake strategy with an explicit backfill policy.

S11-T04 Recompute the affected historical business dates when late orders arrive. Include related items, payments, returns, and deliveries as a consistent change set.

S11-T05 Stage an output, validate it, then publish it. Prevent readers from observing a half-finished daily report; state the single-writer limitation of your local-file approach.

S11-T06 Stop a run after staging but before publication. Restart it and prove there are no extra rows, missing rows, or mixed report versions.

S11-T07 Compare a full rebuild of baseline plus all seven batches against your incremental result. Check keys, row counts, totals, and grouped results, not only one grand total.

S11-T08 Backfill a chosen earlier month while preserving unrelated periods. Write down the exact inputs and code version needed to repeat that backfill.

What to hand over. Incremental pipeline, run ledger, rerun evidence, full-rebuild comparison, and a backfill procedure.

Done when. 350 late orders are considered; some may still fail ordinary quality checks. Reprocessing every batch changes neither keys nor totals. Failed runs never mark data as published.

Be ready to explain. Why is the latest business timestamp usually not a complete ingestion checkpoint?

Sprint 12 | Keep customer history correctly

Advanced / Planning estimate: 12-18 focused hours

The request. A customer moved cities. Finance wants old orders to keep the correct historical city, while support wants the current city.

Use these inputs. Customer baseline and customer_cdc; C000900 late correction; conflicting changes fixture.

What you learn. Change data capture, event ordering, SCD Type 1 and Type 2, temporal joins, tombstones and late corrections.

S12-T01 Explain insert, update, and delete events using the supplied change records. Treat after as the authoritative full new row; before is context and is not guaranteed to be the immediately previous version.

S12-T02 Build a current-customer view and a history table with valid_from, valid_to, current flag, and deletion state. Use half-open validity intervals.

S12-T03 Deduplicate change_id replays. Order effective versions by effective_ts and source_sequence, and document the response to equal sequence values with different payloads.

S12-T04 Apply the new customer insertion and ten soft deletes. Keep historical sales readable while the current view marks the deleted customers inactive.

S12-T05 Handle C000900's February, June, then late April change. Repair the middle historical interval without replacing newer history with an older arrival.

S12-T06 Join orders to the customer version valid at order time. Rebuild historical store/city/customer summaries affected by retroactive changes.

S12-T07 For the LATE_DIMENSION fixture, park an unresolved fact or use a documented unknown member, then repair it when its customer arrives. Do not invent a real identity.

S12-T08 Test for overlapping intervals, more than one current record, empty intervals, lost history, duplicate events, and ambiguous conflicting changes.

What to hand over. Current and historical customer tables, temporal joins, CDC audit records, and late-correction tests.

Done when. C000900 has boundaries at signup, February 1, April 1, and June 1. Deleted customers retain historical versions. CDC reruns do not create extra versions.

Be ready to explain. What is the difference between when a change became true and when your system learned about it?

Sprint 13 | Work through advanced SQL and database behavior

Advanced / Planning estimate: 12-18 focused hours

The request. An analyst asks difficult questions, and the team needs to know when a database operation is safe and efficient.

Use these inputs. Gold facts, categories hierarchy, relational edge cases and a disposable DuckDB database.

What you learn. Advanced CTEs, recursive queries, sets, conditional aggregation, grouping sets, transactions, constraints and plans.

S13-T01 Use a recursive query in DuckDB to return the category hierarchy and depth. Identify orphan parents and describe how to protect against cycles in a future feed.

S13-T02 Find products bought by every active purchasing store in a chosen period. Define the store population first and compare two query strategies.

S13-T03 Find customers who bought from category A but never category B using anti-join or existence logic. Use the null fixture to explain why NOT IN needs care.

S13-T04 Calculate top-category revenue share, Pareto contribution, median order value, and return-rate percentiles. State whether percentiles are exact or approximate.

S13-T05 Produce daily, monthly, and all-store summaries using grouping sets or an equivalent supported design. Distinguish subtotal NULLs from missing business values.

S13-T06 Create constrained tables in a disposable database. Demonstrate a transaction that succeeds and one that rolls back completely after an error.

S13-T07 Inspect query plans for selective filtering, joining, and aggregation. Explain which SQL expressions make filtering harder for the engine to optimize.

S13-T08 Compare Spark SQL and DuckDB implementations of three reports. Record actual dialect differences instead of assuming every statement works in both.

S13-T09 Local extension: install PostgreSQL only if you want server-database practice. Compare indexed lookups, transaction isolation, locks, and JDBC batch loading. Report observations separately from Spark partitioning.

What to hand over. An advanced SQL portfolio, transaction experiment, plan notes, and a dialect comparison.

Done when. Transactions leave no partial committed test data after rollback. Set-based queries handle nulls and empty populations deliberately. Unsupported syntax is not silently replaced with a different question.

Be ready to explain. Why are an index, a Spark partition, and a Parquet partition three different things?

Sprint 14 | Build tests that catch business mistakes

Advanced / Planning estimate: 10-16 focused hours

The request. Before a change is released, show that it does not silently change revenue or lose customer history.

Use these inputs. Hand-check files, challenge packs, clean representative subsets and all prior transformations.

What you learn. Unit, integration, contract, reconciliation, regression and property tests; continuous integration.

S14-T01 Create tiny input datasets yourself for each transformation. Cover valid, empty, null, malformed, duplicate, conflicting, and boundary values.

S14-T02 Test the hand_check money calculations independently from your production transformation. Do not generate expected values using the same function under test.

S14-T03 Test schema, key uniqueness, parent-child links, allowed values, row accounting, payment reconciliation, and SCD interval rules.

S14-T04 Compare SQL and DataFrame versions on a deterministic subset, then on the full Gold aggregates. Normalize sort order and timestamp types before comparison.

S14-T05 Test a complete baseline-to-Gold run in an isolated temporary location. Check that source files are unchanged and temporary outputs are cleaned safely.

S14-T06 Introduce a deliberate business bug, such as counting failed payments, and prove your tests fail for the right reason.

S14-T07 Build a local continuous-integration equivalent that runs formatting/lint checks and relevant tests on demand. An online CI service is optional, not required.

S14-T08 Record acceptance thresholds: zero unresolved key duplicates in published facts; exact money equality; expected known-source rejects; and explicit handling of newly observed quality problems.

What to hand over. A test suite you wrote, representative fixtures, test report, and release checks.

Done when. The intentional bug is caught. A failing required check blocks publication. Tests detect incorrect content, not merely that an output directory exists.

Be ready to explain. Which checks belong in every run, and which are better suited to a scheduled deeper audit?

Sprint 15 | Schedule, monitor, and operate the pipeline

Advanced / Planning estimate: 12-18 focused hours

The request. The team needs a dependable daily process and enough information to fix failures without reading every source file.

Use these inputs. Your jobs, run ledger, quality metrics, and seven incremental batches.

What you learn. DAGs, dependencies, retries, parameters, structured logs, observability, SLAs and secrets.

S15-T01 Draw the dependency graph and turn it into a local scheduled workflow. Begin with a lightweight local runner; use local Airflow as an extension after the same jobs work independently.

S15-T02 Pass run ID, batch date, input/output roots, and environment settings into jobs. Do not make jobs depend on a notebook being open.

S15-T03 Set bounded retries for temporary failures and stop immediately for contract violations. Add a single-writer guard to prevent overlapping publications.

S15-T04 Capture input/output counts, rejection counts, run duration, freshness, publication version, and failure reason in structured logs or a metrics table.

S15-T05 Define a demo service target: valid received batches become reportable within your measured local budget; finance publication waits for required sources. Choose a budget after measuring, not by guessing.

S15-T06 Simulate a missing payments file, a permission failure, and a late source arrival using disposable copies. Describe when reports should remain on the previous complete version.

S15-T07 Write a runbook a colleague could follow: identify a failed batch, inspect evidence, fix the cause, restart safely, backfill if needed, and confirm recovery.

S15-T08 Mask email and name in logs and analyst outputs. Keep configuration separate from secrets. Create a local alert file or dashboard; sending real messages is unnecessary.

What to hand over. A scheduled local workflow, metrics and logs, service-target note, and operational runbook.

Done when. One failed dependency blocks downstream publication. A rerun is safe. Logs identify the failing job and batch without revealing direct identifiers unnecessarily.

Be ready to explain. What is the difference between a job that succeeded and data that is actually ready for a business user?

Sprint 16 | Make Spark faster with evidence

Advanced / Planning estimate: 12-18 focused hours

The request. The pipeline works, but one join is slow and some tasks take much longer than others. Explain why before changing settings.

Use these inputs. Full order_items, products, customers, events and your Gold builds; hot product P00001.

What you learn. Execution plans, shuffles, partition pruning, pushdown, join strategies, AQE, skew, caching, file sizing and UDF costs.

S16-T01 Record an unchanged baseline run: elapsed time, Spark configuration, data size, task count, shuffle bytes, spill, and the slowest stages.

S16-T02 Inspect a physical plan and identify scans, filters, exchanges, joins, and aggregations. Explain which changes should reduce work.

S16-T03 Compare CSV and Parquet reads, selected columns versus all columns, and filtering before versus after a join. Verify results remain equal.

S16-T04 Compare a small-dimension broadcast join with a shuffle join and observe the actual executed plan. Treat join hints as requests, not proof of the chosen strategy.

S16-T05 Measure the P00001 hot key. Demonstrate skew in a relevant workload; compare adaptive handling and a carefully designed salting experiment. Prove salting preserves totals.

S16-T06 Experiment with shuffle partitions, repartition versus coalesce, and output file sizes. Avoid producing one file for the whole dataset as a default strategy.

S16-T07 Cache only a reused intermediate result and measure the benefit and memory cost. Compare built-in expressions with a small Python UDF experiment; explain serialization overhead.

S16-T08 Repeat meaningful measurements and present a before/after table. If the dataset is too small to show a benefit, report that honestly; optionally create a larger local benchmark with correctly remapped keys.

What to hand over. A performance investigation, plan evidence, measurements, and a justified tuning decision.

Done when. Correctness is unchanged. You name the bottleneck and the metric improved. No claim about multi-node scale is based solely on a laptop benchmark.

Be ready to explain. Why can adding partitions, caching more, or broadcasting a table make a job slower?

Sprint 17 | Process events as they arrive

Advanced / Planning estimate: 12-18 focused hours

The request. Marketing wants a rolling event report that can resume after a restart and has a clear policy for late data.

Use these inputs. challenge_packs/stream_sequence first; then sanitized event files replayed in controlled order.

What you learn. Structured Streaming, micro-batches, checkpoints, event-time windows, watermarks, state and delivery guarantees.

S17-T01 Create an empty local inbox and copy one supplied stream_sequence file into it at a time. Use one file per micro-batch and wait for each batch to finish.

S17-T02 Read events with an explicit schema. Start with a stateless cleaned-event output before adding stateful aggregation.

S17-T03 Build 10-minute event-time counts with a one-hour watermark. Observe the maximum event time, watermark, input rows, state rows, and output after every batch.

S17-T04 Deliver the intentionally old W4 event only after the watermark has advanced past its window. Compare it with the still-in-range W5 event. Explain that watermark progress depends on processed event times, not the wall clock alone.

S17-T05 Test replayed event IDs in a separate deduplication query with a documented state-retention policy. Avoid assuming every chain of stateful operations is supported in your Spark version.

S17-T06 Stop and restart using the same checkpoint and prove expected recovery. Then use a separate disposable output to demonstrate the consequences of a new checkpoint.

S17-T07 Compare update and append output behavior. Explain when final window results appear and why an idle stream may not immediately close every recent window.

S17-T08 Design an idempotent batch sink if using foreachBatch. State the guarantees of your actual source, checkpoint, and sink together; do not label every pipeline exactly-once.

What to hand over. A local streaming demo, batch-by-batch observation sheet, restart test, and late-event policy.

Done when. The tiny ordered fixture is tested before bulk historical events. Checkpoints survive restarts. Duplicate and late-event behavior is explained with observed progress.

Be ready to explain. What can still go wrong if the computation is replayable but the sink appends duplicates?

Sprint 18 | Handle table updates, privacy, and recovery

Advanced / Planning estimate: 12-18 focused hours

The request. The platform must support safe corrections and recovery, and customer data should only appear where it is needed.

Use these inputs. Your curated tables, customer CDC and disposable copies for recovery drills.

What you learn. ACID table formats, merges, schema changes, version history, rollback, concurrency, retention and access boundaries.

S18-T01 Add a local Delta Lake experiment using a release compatible with the chosen Spark version. Keep the existing plain-Parquet result as a correctness reference.

S18-T02 Apply deduplicated inserts, updates, and deletes to a current-customer table. Explain why a merge source with conflicting rows needs a policy before the merge.

S18-T03 Read an earlier table version, compare it with the current version, and rehearse a controlled restoration in a disposable location.

S18-T04 Test an additive schema change and a rejected incompatible change. Record which application versions can read each table version.

S18-T05 Simulate competing writers in a small isolated test and inspect the actual conflict behavior. Explain why ordinary file overwrite does not provide the same transactional contract.

S18-T06 Design a data-access boundary: raw restricted area, curated analyst area, and safe logs. Explain why hashing an email is not automatically anonymization.

S18-T07 Write a deletion-request procedure covering current tables, history, exports, raw retention and backups. This is a training design exercise, not a claim of legal compliance.

S18-T08 Set explicit retention and recovery objectives for the demo. Test restoration from a saved backup and explain how cleanup of old table files affects time travel.

What to hand over. A local ACID-table lab, version/recovery evidence, privacy design, and retention decision record.

Done when. Merges are rerunnable and reconcile with the reference result. Destructive cleanup is tested only on disposable copies. Recovery limits are stated clearly.

Be ready to explain. Why can a logical deletion, a physical deletion, and an expired backup have different timelines?

Sprint 19 | Deliver and defend the complete system

Senior capstone / Planning estimate: 16-24 focused hours

The request. You own this project. Demonstrate it to finance, an analyst, and an engineering reviewer, then defend the tradeoffs.

Use these inputs. All sources, all accepted outputs, tests, incidents, metrics and design decisions.

What you learn. Ownership, architecture, stakeholder communication, incident response, release management and capacity planning.

S19-T01 Start from a clean output location and demonstrate baseline ingestion, quality checks, Gold publication, seven incremental batches, and a safe rerun.

S19-T02 Present a compact report answering the ten acceptance questions in this book. State the data cutoff, money unit, exclusions, and uncertainty for each result.

S19-T03 Run a surprise incident chosen from the failure-drill list. Diagnose the cause, protect published data, recover safely, and write a short incident report.

S19-T04 Have a reviewer challenge your metric definitions, order eligibility, SCD history, late-data handling, partitioning, and privacy boundaries. Record and resolve the findings.

S19-T05 Estimate what changes at 10 times and 100 times the input size: CPU, memory, disk, file counts, network, state, and operating cost. Separate measured evidence from estimates.

S19-T06 Design a possible future cloud migration without using any cloud service. Map local storage, compute, orchestration, catalog, secrets and monitoring to capabilities, not just product names.

S19-T07 Package a release with dependency versions, instructions, tests, sample reports, runbook, architecture decisions and limitations. Keep raw data and secrets out of the source repository.

S19-T08 Deliver a 15-minute demonstration and a two-page handover. Include one optimization you rejected and one failure you learned from, with evidence.

What to hand over. A reproducible release, business demonstration, operations handover, incident report, and architecture defense.

Done when. A reviewer can reproduce the project from your instructions. Full and incremental results reconcile. You can recover from failure and explain limitations without hiding them.

Be ready to explain. If a manager says revenue is wrong today, what evidence do you inspect first, and why?

Scenario coverage

Part 1 / learn from realistic data, not only clean happy paths

Scenario	Where to find it	Sprints	Expected lesson
Exact duplicates	Customers, orders, items, events, CDC	5, 6, 11, 12	A replay does not change the final business answer.
Missing city and invalid email	customers	2, 6	Optional attribute failures do not delete usable customers.
CSV quoting	customer names containing commas and quotes	1, 5	The parser preserves a single field and record.
Invalid timestamp / missing customer	orders	4, 6	Parsing and mandatory-key problems remain traceable.
Unknown customer / product	C999999, P99999	3, 6	No silent inner-join loss; affected orders are ineligible.
Negative item quantity	order_items	6, 8	Invalid rows cannot lower reported sales accidentally.
Malformed JSON	21 web-event lines	5, 7	Every received line is accounted for.
Nested and empty arrays	web_events.cart	7	Exploding does not change event-level metrics.
Anonymous visits and abandoned carts	web_events	7, 9	They remain valid sessions, with no invented customer.
Payment retries and pending/cancelled orders	payments and orders	3, 8	Failed charges and ineligible orders are not sales.
Partial returns and refunds	returns and REFUND payments	8	Units, line amounts and cash reconcile.
Late and incomplete delivery	shipments	8	Undelivered orders are separate from delivered-rate denominators.

Scenario coverage

Part 2 / advanced changes and isolated incident packs

Scenario	Where to find it	Sprints	Expected lesson
Inventory zeros and coverage gaps	June subset of inventory	9, 10	Missing coverage is not treated as zero stock.
A hot join key	P00001, intentionally about 35% of item selection	16	Measure actual skew and preserve correctness when tuning.
Late historical orders	350 rows in the seven later batches	11	Earlier business dates are repaired when needed.
Additive event schema	app_version from July 4 batches	7, 18	Older and newer events coexist.
Insert, soft delete, update, no-op, late correction	customer_cdc, especially C000900	12	Current and historical states remain consistent.
Conflicting same-sequence changes	conflicting_customer_changes.jsonl	6, 12	Ambiguity is surfaced rather than silently resolved.
Header-only and wrong-header input	orders_header_only.csv, orders_wrong_header.csv	5, 15	Zero rows and broken contract are different outcomes.
Future time and wrong JSON field type	schema_break.jsonl	7, 14	Invalid types/timestamps are visible and handled by policy.
Over-refund, excess return, bad money	amount_and_quantity_errors.jsonl	6, 14	Business limits are tested on tiny explicit cases.
Null exclusion, ties, missing dates, fan-out	relational_edge_cases.jsonl	3, 9, 13	Queries answer the intended business question.
Late dimension and repeated local clock	relational_edge_cases.jsonl	7, 12	Unknown identities and time ambiguity are handled explicitly.
Controlled stream lateness and replay	stream_sequence/01 through 07	17	Observed watermark and deduplication behavior is documented.

The coverage is deliberately broad, not exhaustive. For missing-file, resource, concurrent-writer and crash scenarios, you create a controlled failure using disposable copies. Do not corrupt the only copy of your work.

Failure drills and review evidence

Practice recovery before calling a system dependable

01. Replay a completed batch under a new filename. Explain whether checksum identity or business identity prevents duplication at each layer.

02. Remove an expected input from a disposable copy. Confirm that an incomplete finance report is not published.

03. Stop a process before staging completes, after staging succeeds, and immediately before the run is marked successful. Recover each case.

04. Deliver the oldest incremental order after the newest batch. Compare the affected historical report with a complete rebuild.

05. Inject a malformed timestamp, a newly unknown status, and a breaking schema separately. Confirm that each failure points to the source record.

06. Change the discount logic incorrectly. Demonstrate that a business test blocks the release.

07. Start two publications for the same output. Demonstrate the local concurrency guard or transactional conflict behavior.

08. Simulate a small disk quota or write failure in a disposable area. Do not intentionally fill your real disk.

09. Copy a stream event twice, pause the stream, restart with the same checkpoint, and inspect what was processed.

10. Restore an older curated backup to a separate location and validate it before switching readers.

11. Write an incident report with impact, detection, timeline, root cause, recovery, and a prevention action. Separate evidence from assumptions.

Your review packet

For every advanced sprint, keep a small input example, the expected outcome, the actual outcome, relevant logs or plan, the rerun result, and a short explanation. Include a failure you fixed, not only a successful final screenshot.

Senior-level review lens

Assess correctness, reproducibility, recovery, performance evidence, data contracts, security boundaries and clarity of tradeoffs. A blocked publication gate cannot be compensated for by a good-looking dashboard.

Optional labs after the core works

Broaden tool experience without losing the main project

API ingestion

Wrap a subset in a local paginated API that you implement. Practice pagination, retries, rate-limit responses, caching identifiers and malformed responses. Keep the original files as the correctness reference.

PostgreSQL and JDBC

Run a local server, create constraints and indexes, inspect plans and locks, and load/extract in controlled batches. Do not treat file-based CDC as proof you have implemented database-log capture.

Kafka and real CDC tooling

After file streaming works, run a local broker and study partitions, offsets, consumer recovery and delivery semantics. Add a local database-log CDC tool only as a separate lab.

Airflow or another orchestrator

Move independently runnable jobs into a local scheduler. Practice backfills, retry limits, task dependencies, scheduling time versus business time, and failure visibility.

dbt or another SQL transformation framework

Rebuild selected Gold SQL models, their dependencies, tests and documentation. Compare maintainability and engine compatibility with your existing approach.

Container packaging

Package a working, version-pinned environment. Check startup memory, persistent volumes and recovery; do not hide manual setup steps inside an unexplained container.

Larger performance experiments

Create your own larger synthetic benchmark only after fixing correctness. Remap every primary and foreign key consistently; simply copying files creates duplicate inputs, not additional business entities.

Cloud and distributed systems design

Describe object storage, managed compute, catalogs, secrets, queues, observability, access controls and cost budgets. A design exercise is not a deployed or load-tested cloud system.

Data dictionary

Part 1 of 7 / these are target meanings; raw values can fail validation

customers

One customer profile per customer_id before exact replays. Required keys must be usable; city and email can have repairable quality issues.

Field	Target type	Meaning
customer_id	Text	Customer business key, such as C000001.
customer_name	Text	Synthetic display name; may contain CSV quotes or commas.
email	Text, optional	Synthetic email; invalid values must be flagged and nullified.
city, state	Text	Customer location. City can be blank, lower-case or padded.
signup_ts	UTC timestamp	Customer creation time in 2024; initial history starts here.
segment	Text enum	NEW, REGULAR or VIP.
is_active	Boolean	CSV true/false; a soft delete later changes current state.
source_updated_at	UTC timestamp	Source update time, not your ingestion timestamp.

categories

One category per category_id. Four root categories and eight child categories.

Field	Target type	Meaning
category_id	Text	Category business key.
category_name	Text	Readable category name.
parent_category_id	Text, optional	Another category_id; empty for a root category.

suppliers

One supplier per supplier_id.

Field	Target type	Meaning
supplier_id	Text	Supplier business key.
supplier_name	Text	Synthetic supplier label.
city	Text	Supplier city.
lead_time_days	Integer	Planning estimate, not an observed shipment duration.

Data dictionary

Part 2 of 7 / these are target meanings; raw values can fail validation

products

One catalog product per product_id. Historical order prices come from order_items, not this catalog.

Field	Target type	Meaning
product_id	Text	Product business key; P00001 is intentionally popular.
product_name	Text	Synthetic product label.
category_id, supplier_id	Text	References to categories and suppliers.
brand	Text	Synthetic brand label.
catalog_price_paise	64-bit integer	Current catalog price in paise.
unit_cost_paise	64-bit integer	Static synthetic reference cost; insufficient for real historical margin accounting.
is_active	Boolean	Current catalog activity. Inactive products can have valid historical orders.

stores

One store or warehouse per store_id.

Field	Target type	Meaning
store_id	Text	Store business key.
store_name	Text	Synthetic location name.
city, state	Text	Location geography.
store_type	Text enum	STORE or WAREHOUSE.

Data dictionary

Part 3 of 7 / these are target meanings; raw values can fail validation

orders

One order header per order_id before exact replays. Initial and incremental records are inserts; customer history changes are in a separate CDC feed.

Field	Target type	Meaning
order_id	Text	Order business key, such as O0000001.
customer_id, store_id	Text	Required references to customer and store; deliberate customer defects exist.
order_ts	UTC timestamp	Order business time; deliberately invalid text occurs in some records.
order_status	Text enum	PENDING, CANCELLED, PAID, SHIPPED or DELIVERED.
sales_channel	Text enum	WEB, APP or STORE.
shipping_fee_paise	64-bit integer	Order-level shipping fee; do not repeat it per item.
order_total_paise	64-bit integer	Sum of line net plus shipping, before returns. No tax component.
source_updated_at	UTC timestamp	Time of the represented state. Keep separate from file arrival.

order_items

One purchased line per order_item_id before exact replays. An order can have several lines for the same product.

Field	Target type	Meaning
order_item_id	Text	Line business key, such as I0000001_1.
order_id	Text	Parent order key.
product_id	Text	Product key; P99999 is an intentional missing product.
quantity	Integer	Purchased units; must be positive after validation.
unit_price_paise	64-bit integer	Price per unit at purchase time.
discount_paise	64-bit integer	Total line discount, bounded by quantity times unit price.

Data dictionary

Part 4 of 7 / these are target meanings; raw values can fail validation

payments

One payment attempt or refund per payment_id. An order can have a failed attempt, a successful charge, and a later refund.

Field	Target type	Meaning
payment_id	Text	Payment-event business key; do not deduplicate by order_id.
order_id	Text	Parent order key.
payment_ts	UTC timestamp	Payment event time; use this for cash-date reports.
payment_type	Text enum	CHARGE or REFUND.
payment_status	Text enum	SUCCESS or FAILED; only SUCCESS affects cash.
payment_method	Text enum	CARD, UPI or NETBANKING.
amount_paise	64-bit integer	Positive magnitude. Payment type determines cash direction.
related_return_id	Text, optional	Return business key for REFUND; empty for CHARGE.

shipments

One shipment per shipment_id in this dataset, at most one per order. Production designs may need split shipments.

Field	Target type	Meaning
shipment_id	Text	Shipment business key.
order_id	Text	Parent order key.
carrier	Text	CarrierA, CarrierB or CarrierC.
shipped_ts	UTC timestamp	Shipment departure time.
promised_delivery_ts	UTC timestamp	Original promised deadline.
delivered_ts	UTC timestamp, optional	Actual delivery time; blank while in transit.
shipment_status	Text enum	IN_TRANSIT or DELIVERED.

Data dictionary

Part 5 of 7 / these are target meanings; raw values can fail validation

returns

One return per return_id, referring to a single item. Current data has at most one partial return per selected order, but your tests must cover cumulative limits.

Field	Target type	Meaning
return_id	Text	Return business key.
order_item_id	Text	Returned item; derive order_id through the item relationship.
return_ts	UTC timestamp	Return event time, after delivery.
return_quantity	Integer	Positive returned units.
refund_amount_paise	64-bit integer	Approved merchandise refund; shipping is excluded.
reason	Text enum	DAMAGED, WRONG_SIZE or CHANGED_MIND.
return_status	Text enum	APPROVED in this source version. Do not assume future statuses are automatically approved.

inventory

One snapshot per snapshot_date, store_id and product_id. Coverage: June 1-30, stores S001-S004, products P00001-P00500.

Field	Target type	Meaning
snapshot_date	Date	Calendar date of the stock snapshot.
store_id, product_id	Text	Composite key components and dimension references.
on_hand_quantity	Integer	Available units, including valid zeros.
reorder_level	Integer	Threshold used for low-stock reporting.

Data dictionary

Part 6 of 7 / these are target meanings; raw values can fail validation

web_events

One event per event_id before exact replays; malformed physical lines have no usable event key. JSON Lines format.

Field	Target type	Meaning
event_id	Text	Event business key. Preserve source record identity for corrupt lines.
session_id	Text	Source session key; some abandoned sessions have no customer.
customer_id	Text or null	Optional logged-in customer ID.
event_type	Text enum	view_product, add_to_cart, checkout or purchase.
event_ts, received_ts	UTC timestamp	When the activity happened and when this export received it.
order_id	Text or null	Present for purchase events only; non-purchase events are not order lines.
product_id	Text	Viewed/primary product; not a full purchased-cart measure.
device	Object	type: mobile/desktop/tablet; os: Android/Linux/iOS.
marketing	Object	source: organic/email/paid_search; campaign: text or null.
cart	Array of objects	Each element has product_id (text) and quantity (integer); can be empty.
schema_version	Integer	1 in baseline/early batches; 2 from July 4 batches.
app_version	Optional text	Added in version 2; absent in version 1, not a parsing failure.

Data dictionary

Part 7 of 7 / these are target meanings; raw values can fail validation

customer_cdc

One source change event per change_id before exact replays. These are synthetic file-based change events, not captured database logs.

Field	Target type	Meaning
change_id	Text	Unique change identity; use to recognize replay.
customer_id	Text	Customer whose state changes.
operation	Text enum	I = insert, U = update, D = soft delete.
effective_ts	UTC timestamp	When the new business state becomes valid.
source_sequence	64-bit integer	Ordering/tie-breaker from the source. Equal timestamp and sequence with different payload is a conflict.
received_ts	UTC timestamp	When the change arrived. Arrival order can differ from effective order.
before	Customer object or null	Context only; not guaranteed to be the immediately prior image.
after	Customer object or null	Full authoritative image for I/U, null for D. Contains customer fields from the customer dictionary.

Type 2 history tracks customer business attributes, not every technical update timestamp. An update that changes no tracked business attributes can be a no-op; still record that the change event was processed. Conflicting events are isolated in a challenge pack, not quietly resolved for you.

Plain-language glossary

Use the vocabulary, but always be able to explain it

Term	What it means
Pipeline	A repeatable set of steps that turns incoming data into a useful output.
Schema / contract	The expected columns, types and rules that producers and consumers agree on.
Grain	What one row means. Decide this before joining or calculating totals.
Primary / foreign key	An identifier for a row or entity, and a reference to an entity in another table.
Bronze / Silver / Gold	Preserved received data, cleaned usable data, and business-ready tables.
Quarantine	A visible holding area for records that failed a rule and need investigation.
Idempotent	Running the same logical input again produces the same business result.
Backfill	Processing an older period again to add missing data or correct previous results.
CDC	A record of inserts, updates and deletes rather than just a complete current snapshot.
SCD Type 2	Keeping dated versions of descriptive information so history stays accurate.
Partition / shuffle	A share of work/data, and redistribution between tasks when the operation needs it.
Skew	Some keys or partitions contain much more work than others.
Checkpoint / watermark	Saved processing progress, and an event-time boundary used by a streaming query for late data/state management.
DAG	A dependency graph that states which jobs must finish before other jobs can begin.
ACID / transaction	A set of database guarantees for safely committing related changes as a unit.
Lineage	The path from original sources through transformations to a report value.
SLA / SLO	An agreement or an internal target for service behavior such as delivery time or availability.
RPO / RTO	How much data loss is acceptable, and how long recovery may take.

Official references and your first session

Use documentation for your chosen version

Apache Spark 3.5.7 installation and requirements

https://spark.apache.org/docs/3.5.7/api/python/getting_started/install.html

Apache Spark DataFrame quick start

https://spark.apache.org/docs/3.5.7/api/python/getting_started/quickstart_df.html

Apache Spark SQL reference

https://spark.apache.org/docs/3.5.7/sql-ref.html

Apache Spark performance tuning

https://spark.apache.org/docs/3.5.7/sql-performance-tuning.html

Apache Spark Structured Streaming guide

https://spark.apache.org/docs/3.5.7/structured-streaming-programming-guide.html

Apache Spark testing guidance

https://spark.apache.org/docs/3.5.7/api/python/getting_started/testing_pyspark.html

DuckDB installation and version selection

https://duckdb.org/install/

DuckDB Python client

https://duckdb.org/docs/current/clients/python/overview

DuckDB transactions

https://duckdb.org/docs/current/sql/statements/transactions

Delta Lake / Spark version compatibility

https://docs.delta.io/releases/

Delta Lake local quick start

https://docs.delta.io/quick-start/

What to do first

Read the complete project question, then do Sprint 00 only. Create your folders, verify your environment, and inspect the hand-check files. In Sprint 01, write your first small file utility. You do not need to understand streaming or SCD history on day one.

What is intentionally not included

There are no ready-made ETL jobs, SQL answers, implementation tests, scheduler definitions, or generator scripts. Tiny expected values and raw-file hashes are provided so you can check your own work.

Source note

Reference environment guidance was checked against the official documentation on 21 October 2026. The assignments and business rules are original specifications for this synthetic training project; they are not claims about a real company.